

Fun Optimizations for eBPF Programs and How to Enable Them

Farbod Shahinfar
Politecnico di Milano
Italy, Milan

Aurojit Panda
New York University
United States, New York

Gianni Antichi
Politecnico di Milano
Italy, Milan
Queen Mary University of London
United Kingdom, London

Abstract

The eBPF runtime relies on a static program verifier to check safety properties of custom programs before they are executed in the kernel. In some cases, the verifier limits compiler optimizations because it cannot reason about the optimized code. Moreover, developers are forced to use costly abstractions to satisfy the verifier. We measure the performance overhead introduced by these restrictions and find that it can halve performance in a compute-heavy benchmark under workload with large number of iterations. Finally, we propose an extension to the compilation process with a post-verification optimization phase to address this limitation.

CCS Concepts

• **Software and its engineering** → **Just-in-time compilers**; **Software performance**; *Operating systems*; • **Networks** → *Network servers*; *Data center networks*; *Programmable networks*.

Keywords

eBPF, Compiler Optimization, JIT Compilers, Kernel Extension, Microkernel Architecture

ACM Reference Format:

Farbod Shahinfar, Aurojit Panda, and Gianni Antichi. 2026. Fun Optimizations for eBPF Programs and How to Enable Them. In *4th Workshop on eBPF and Kernel Extensions (eBPF'26)*, September 29–October 02, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3837779.3838161>



This work is licensed under a Creative Commons Attribution 4.0 International License.

eBPF'26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2911-9/2026/09

<https://doi.org/10.1145/3837779.3838161>

1 Introduction

eBPF is the Linux kernel's programmability technology with many applications including packet processing [7, 46], system monitoring [6], process scheduling [14], out-of-memory management [2], and more [15].

Performance is a key requirement for eBPF applications, and like any software, compiler optimizations are fundamental for having efficient programs [8]. Previous works have shown that the eBPF compiler can be improved [34, 48, 56]. But they all are limited by the eBPF verifier ability to reason. It is because the verifier relies on symbolic analysis of code paths, and compiler optimizations collapse or restructure those paths in ways that the verifier cannot follow.

In this work we show that this limitation forces both the compiler and the developer to make choices that sacrifice performance. The compiler misses optimizations such as unrolling loops, which can halve the performance of a simple compute-heavy benchmark (§2.1) in cases with large iterations. Furthermore, some common programming constructs, e.g., unbounded loops are entirely inexpressible in eBPF code, and developers must instead use functions that are provided by the runtime and recognized by the verifier (e.g., `BPF Iter` [38]). Calling these functions adds overheads, for example, we found the loop API can reduce performance of the compute-heavy benchmark by 88%.

Despite the overhead, the eBPF verifier is an important component of the eBPF system and removing it is not acceptable. Instead, our insight for addressing the limitation is that after the verifier determines the safety of a program, we can simply continue optimizations and trust it remains safe. This is true because a correct optimizer will not change the behavior of the program. Later, we present a proposal for extending eBPF runtime to support post-verification optimization phase. Our contributions are:

- (1) We share scenarios in which eBPF verifier prevents code optimization (§2.1 & §3) and discuss how developers have to use less performant programming abstractions to meet the verification requirement (§3.1).
- (2) We perform a micro-benchmark to measure the effect of these overheads on eBPF programs (§2.1).

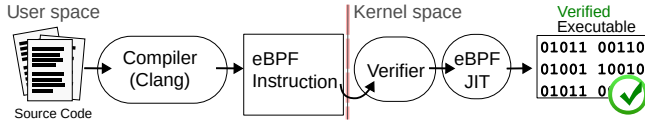


Figure 1: A high-level eBPF source code is compiled to eBPF instruction (bytecode), then it is verified and translated to native instruction to into executable form.

- (3) To address the challenges, we propose an extension of the eBPF compiler pipeline based on users’ needs through an optimization phase that runs after the verifier (§4). We discuss the attributes of this design (§5).

2 eBPF Compilation: A New World

An eBPF program passes through a compiler, a static verifier, and an in-kernel just-in-time compiler (JIT) before becoming executable code [15]. We believe this pipeline is too restrictive for the compiler optimizations because the verifier cannot easily reason about the compiler-introduced transformations and has difficulty proving its correctness.

Figure 1 shows the compilation process in more detail. Initially, a compiler (e.g., Clang) processes the source code of the program and compiles it to eBPF instruction set architecture (ISA). During this process, it applies optimizations such as constant propagation, and dead-code elimination. The resulting eBPF program is then passed to the kernel, where the eBPF verifier first verifies its safety, and later, the eBPF JIT translates the program from eBPF ISA to the host’s CPU ISA to produce executable code [15].

In this process, most compiler optimizations happen in userspace during initial compilation from source code to eBPF ISA. Unlike the initial compilation, the JIT is conservative with optimizing the program and primarily performs translation to avoid changing the program’s behavior and negating the verifier’s work. A potential bug in the code modifications after verification may introduce unsafe behavior that goes unnoticed [47].

Experimental setup. We argue that program verification during the eBPF compilation pipeline leads to inefficient executables, and demonstrate this through benchmarks performed on machines from Cloudlab [12] (c6525-100g machines from Utah cluster). Each machine has AMD EPYC 7402P processors and runs Ubuntu 24.04 with Linux kernel version 6.8.0. The eBPF programs were compiled with Clang version 19 (unless explicitly specified). We used `-O2` optimization flag¹ in all cases.

¹Compiling eBPF programs with `-O2` is the de-facto approach as `-O0` will avoid code transformations required for program to pass the verifier and `-O3` is considered unstable.

```
1 for (; i < CONSTANT_BOUND && i < repeat; i++) {
2     c = a + b; a = b; b = c;
3 }
```

Figure 2: A compute-heavy loop. It is bounded by a variable (i.e., `repeat`) and a constant value (i.e., `CONSTANT_BOUND`). The constant bound helps both verifier and the compiler to reason about loop.

2.1 Look! Code Optimization Challenge

To benchmark the eBPF compilation pipeline we use a program calculating the N^{th} Fibonacci number. The code is compute bound which simplifies considerations with memory stalls. The faster a program runs, the more efficient its executable is.

Figure 2 shows our example. The interesting part affecting compile time optimization is the two clauses in the for-loop’s termination condition. One checks against a constant value (known at compile time) and the other checks a variable upper bound.

If only the variable bound is checked, then the verifier cannot reason about the loop termination and the compiler also cannot reason about the benefit of unrolling. But, by adding the constant upper bound, both the verifier and the compiler can reason about the loop, and therefore the program passes the verifier, and the compiler can decide whether unrolling the loop yields better performance.

Loop unrolling is important in our example. It amortizes the cost of backward jump over multiple iterations. But surprisingly, Clang decides not to unroll the loop, and when explicitly instructed to unroll (using annotations), the compiled program does not pass the verifier.

Looking more closely, Clang generates a program that first computes the minimum of constant and variable upper bound, and later, it only does one comparison against that value in each iteration which reduces one check per iteration.

The verifier fails to reason about the loop termination after this code transformation, thinking the loop can possibly repeat more than allowed budget². This means the verifier is imposing performance overhead on our program by restricting the compiler and the shape of the code it can generate.

To measure this overhead, we implemented loop unrolling in eBPF assembly (`Unroll-1` scenario) by simply repeating the entire instructions (including both checks on the constant and variable bound) in the loop N times. The implementation passes the verifier.

We compare its performance against the baseline (without unrolling) under three workloads. Each workload is a series of randomly selected numbers for Fibonacci computation

²The verifier error message is: BPF program is too large. Processed 1000001 insns

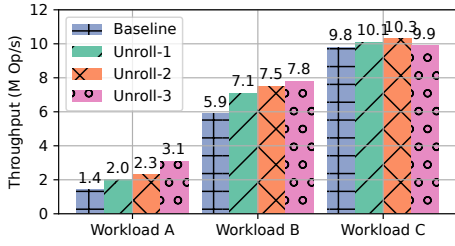


Figure 3: Benefit of unrolling 8 times the loop in Figure 2 using different methods. Baseline is Clang without loop unrolling as it does not pass verification.

within different ranges: **A.** large (0–1024), **B.** medium (0–128), and **C.** small (0–16).

Figure 3 shows the result. Unrolling always performs better than the baseline, and we observed 3% to 42% increased performance, which highlights the overhead of deciding not to unroll loops because the code generated by the compiler (i.e., Clang) is rejected by the verifier.

Inspired by the benefit of amortizing the cost of jump instruction, we further optimized the code by relaxing the comparison with constant upper-bound such that bound check is performed once every N iteration (Unroll-2). This approach further improved performance by 2% to 15% compared to Unroll-1 scenario (Figure 3). This optimization shows the benefit of reducing the number of branches in the body of the loop which is approximately what Clang tried to perform (Clang completely removed the second check while Unroll-2 performs it less often).

We further restructured the code to create paths that perform fewer branches (if possible one for every N iteration) at the cost of more computation (Unroll-3). This doubles performance compared to the baseline in workload **A**, and is 34% faster than Unroll-2. Under workload **B**, it is 4% faster than second best unrolling method (Unroll-2). But, for workload **C** this aggressive optimization performs –4% less than other best unrolling method.

This case highlights the importance of optimizing the structure of the code (in this case algorithm to unroll the loop) based on workload characteristics (in this case based on the number of loop iterations). We have two takeaways from this experiment:

- (1) Optimizing programs requires flexibility because the best performing code depends on many factors including program workload, as we saw with different unrolling methods performance varied significantly.
- (2) The eBPF verifier is restricting the compiler optimizations because it requires certain code structures to reason about safety properties. This restriction is not just a burden for developer when writing code [45], but also has a performance overhead.

But, the compiler optimizations are required for a program to meet the eBPF verification criteria and satisfy the verifier. Therefore, the optimizations are not always fighting with the verifier, but on the contrary, they help with it. In the next section we explore this in more detail through examples.

3 Verifier strikes back

Compiler optimizations are crucial for programs to meet the eBPF verifier’s requirements because they simplify and structure the code. Some examples of how a compiler helps with verification include:

- (1) Compiler removes extra memory accesses and prunes redundant paths. This enables complex programs to fit the complexity limits of the verifier [34].
- (2) The verifier cannot reason about values stored on a map³ (e.g., configurations) since they are volatile and an external component such as another eBPF program can change them. The compiler places these values in registers reducing memory accesses and enables the range tracking algorithm to reason about them.
- (3) In Figure 2, if the compiler knows the number of iterations, it can compute the Fibonacci number and replace the loop with its result⁴. This will greatly simplify the program complexity and assists the verifier.

But compiler optimizations are not always helpful for the verifier. In contrast, they may also shape the code such that the verifier is unable to understand it. The example of loop unrolling in §2.1 showed how the loop termination condition was not understood by the verifier after optimization.

To understand what code transformations a compiler does and how they can help or hinder verification, we looked at the list of code transformation passes from LLVM compiler [19] and highlighted those that in our opinion assist with verification and those that may cause the verification to fail in Table 1. Next, we discuss additional examples of how the verifier prevents common optimizations.

Rejections despite safe specification. Figure 4 shows a program that reads 20 bytes (constant value known at compile time) from the packet and performs some calculation. Although in the source code of the program there are checks for each read operation to be within the correct memory address range, the kernel rejects the program with the verifier’s complaint about potential out of range memory access. Since the code is performing the memory range checks, we suspected that the verifier’s inability to reason about the safety should be due to compiler optimizations. Exploring versions of Clang from 17 to 22 (from 2023 to the latest release at the

³A map is a data-structure allocated by eBPF runtime which can be shared among multiple eBPF programs and userspace processes.

⁴In our test, Clang removes the Fibonacci loop by evaluating it at compile time only if less than 101 iterations are required.

Table 1: Description of some LLVM compiler passes and our opinion on if they help or hinder verification [19] (✓: safe, ♣: may be unsafe).

Category	LLVM Passes	Description	Safe?
Dead Code Elimination	dce, adce, dse loop-deletion globaldce	Remove instructions, basic blocks, loops, or globals whose results are never used, shrinking the program and simplifying control flow; helps verification by reducing code before the kernel’s own dead-code-elimination pass [1]	✓
Argument Manipulation	argpromotion deadargelim	Rewrite function signatures by promoting by-reference scalar arguments to by-value and deleting unused parameters, which simplifies call graphs	✓
Code Motion	block-placement sink, licm	Reorder or move instructions (e.g., hoisting loop-invariants, sinking pure ops, laying out hot blocks) to reduce redundant work; Can result in code-structure that verifier does not understand	♣
Instruction Simplification	instcombine aggressive-instcombine reassociate	Search common patterns and replace or reorder them to optimize or create opportunity of further optimization. The transformations are safe, but may generate a program that is hard for the verifier to reason	♣
Control Flow Reasoning	jump-threading simple-loop-unswitch	Reorders the branches to avoid unnecessary operations and reduce branches. These may challenge the verifier in two cases: 1) creating paths that it can not reason about 2) exceed the eBPF complexity limit for number of jump (8 K) and maximum depth of execution path (1 M)	♣
Loop Optimization	loop-fusion loop-unroll loop-unroll-and-jam	Transform loops (fusing, unrolling, and jamming) to improve locality or reduce branch overhead; these can significantly increase code size and per-path work, risking eBPF instruction-count limits and more complex verifier reasoning	♣
Expression Evaluation	sccp, ipccp, gvn	Perform sparse constant propagation and global value numbering to replace values with constants, prune unreachable paths, and merge redundant computations, generally simplifying the IR and reducing the load on the eBPF verifier	✓

```

1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *) (p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }

```

Figure 4: The program fails the verification because the verifier cannot reason about memory address bounds after optimizations.

```

1 r4 = r3          ; r4 = data_end
2 r4 += 0x1        ; r4 = data_end + 1
3 r1 = r2          ; r1 = data
4 r1 += 0x1        ; r1 = data + 1
5 ; ...
6 r4 = r2
7 r4 -= r1          ; r4 = -(pkt size)
8 if r4 > -0x15 goto +0x20 ; --> exit
9 r4 = 0x0
10 r5 = r2          ; <--
11 ; ...
12 if r4 != 0x14 goto -0x7 ; -->

```

(A)
(B)
(C)
(D)

Figure 5: eBPF program for Figure 4 compiled with Clang v19.

time of writing), we found that in all cases the program is rejected.

To understand why, look at Figure 5 that shows the generated eBPF instructions when compiled with Clang v19. The

program adds 1 to the pointers of the beginning and end of the packet (A), and later calculates the size of the packet using pointer arithmetic (negative of the size in this case) (B). The size of the packet is checked to be at least 20 bytes (C). Checking the packet size makes redundant all the other bound-checks performed inside the loop. Consequently, if the packet is large enough, the program reads the first 20 bytes of the packet and performs calculations without the checks before read operations (D).

The verifier rejects this program with error “pointer arithmetic on pkt_end prohibited”; directly addressing line 2 of Figure 5 which is adding 1 to a pointer. This is the case for versions 17 to 19 of Clang.

With Clang v20 to v22, the compiler avoids the pointer arithmetic (the code is different from Figure 5 but the logic is similar to it). Regardless, the program is still rejected because the verifier is unable to reason about the correctness of bound check algorithm performed.

More specifically, the compiler has recognized that all paths in which the packet is smaller than 20 bytes have the same effect (the program stops) and has simplified the program by checking the packet size. The check makes further bound checks for memory access redundant and the compiler removes them. The verifier only sees the optimized version

of the program and fails to understand the reasoning especially the relation between pointer arithmetic and packet size (Ⓔ in Figure 5). As a result, it rejects the program.

Confusing Upper Bound. In the next example we tried to optimize the loop in Figure 6a by hoisting the bound check out of the loop. Figure 6b is the optimized version. It computes the maximum offset accessed into the buffer by finding the minimum of a variable and a constant (line 2 of Figure 6b) and performs the check before the loop. When the verifier is processing the optimized program, it can deduce that the result of the minimum computation will be less than or equal to the constant value, but it cannot reverse the reasoning to understand that the minimum was computed because it is the upper bound of the loop, and it is the maximum offset into the buffer. Consequently, the verifier rejects the program thinking an invalid memory access may happen⁵.

3.1 Developer’s Dilemma

It is not just the compiler that tries to generate verifier-friendly eBPF programs; developers also write code with verification restrictions in mind [45]. To help developers write code that satisfies the verifier, eBPF has implemented a special API for common operations that are important for different applications but hard for the verifier to reason about. Some examples are:

- The BPF Iterator facilitates unbounded loops to walk list of existing kernel objects (e.g. sockets) [2, 38].
- The Arena improves programmability and enable implementing custom data structures [13, 49].
- The Dynptr [25] was introduced for safe access to kernel owned buffers (e.g., packet data from `sk_buff`) or objects (e.g., files)
- The `bpf_loop` [9] was introduced to allow more complicated loops.

For example, `bpf_loop` separates the loop control-flow and application logic such that the control-flow of the loop (and its termination condition) is implemented in the runtime and the logic is implemented by the developer and passed to the API as a callback function. This separation allows developers to explore variable size loops, because the verifier knows that the runtime will guarantee the termination, so it only explores the safety of the callback function [15].

Unfortunately, these abstractions (i.e., `bpf_loop` and others) are not zero-overhead which forces developers to make trade-off between functionality and performance. To show the overhead of eBPF runtime abstractions, we explored the loop helpers using the same Fibonacci benchmark and

workload as in §2.1. We considered three methods for implementing a loop in eBPF: a normal for-loop, the `bpf_loop` helper, and the BPF iterator.

Figure 7 shows the result. The versions that uses flexible loop API under heavy workload performs down to −88% less operations per second. The specialized loop API are less performant because they do extra function call/return operation per each iteration, which also may increase the pressure on active registers and waste time moving data.

Our takeaway is that the API allows developers to describe their application logic and pass the verification. But, they impose overhead affecting efficiency.

3.2 Empire of Verification

We showed that eBPF compiler pipeline encounters obstacles attempting to satisfy the verifier while simplifying programs to generate efficient code. The inability of the verifier to understand the optimization logic (especially because the optimized code can have less information about the behavior of the program) imposes hidden performance overhead.

But, removing the verifier is not an acceptable solution, because the kernel is running custom programs and it requires some guarantees to make sure they do not crash, corrupt, or hang the kernel [15, 16]. Using a static verifier allows the kernel to use low overhead techniques for sandboxing the programs [16, 41] instead of relying on approaches such as data-copy and context switching.

Finally, the users of eBPF applications also expect the safety guarantees as business requirement. As kernel extension are considered to have the risk of complete system crash when an error occurs and without strong guarantees the risk is evaluated to be too high [11, 18].

4 Return of the Optimizer

To address the problem we rely on two observations:

Observation 1: Performing the optimizations after verification will address the challenge mentioned in §3 as there will be no code structure restrictions.

Observation 2: Although kernel can not reason about the safety of a program modified after verification, a correct optimizer will maintain the safety properties of the program when generating a more efficient executable [8].

Placing these observations together, we can extend the eBPF compilation pipeline with a phase that runs a post-verification optimizer (PVO) to generate more efficient executables that are safe.

An immediate question is *how can the kernel trust a post-verification optimizer?* Unfortunately, proving the correctness of optimizing compilers is hard [28], and it is unrealistic

⁵The verifier error message: invalid access to packet, off=0 size=1, R5(id=0,off=0,r=0); R5 offset is outside of the packet

```

1 for (i = 0; i < n_bytes && i < CONSTANT; i++) {
2   // bound check every access to the buffer
3   if ((void *) (p + i + 1) > data_end)
4     return -1;
5   uint8_t tmp = p[i];
6   // ... rest of computation
7 }

```

(a) Passes the verifier check

```

1 // Check once for every iteration
2 size_t X = min(n_bytes, CONSTANT);
3 if ((void *) (p + X) > data_end)
4   return -1;
5 for (i = 0; i < n_bytes && i < CONSTANT; i++) {
6   uint8_t tmp = p[i];
7   // ... rest of computation
8 }

```

(b) Does not pass the verifier check

Figure 6: Two implementations of a program that reads from a buffer. The verifier accepts the first but not the second although they are equivalent. Figure 6a is performing bound-check before each read, while Figure 6b computes the bound of accessed memory and checks once. The verifier cannot reason about the accessed memory because of variables that are unknown before runtime.

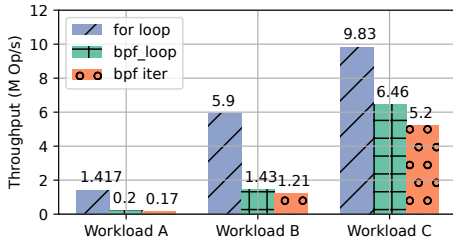


Figure 7: Using flexible eBPF loop APIs decreases throughput down to ~88% when calculating Fibonacci number.

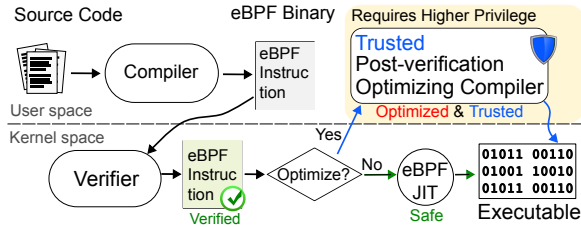


Figure 8: A post-verifier optimizer (PVO) receives verified programs from the kernel and compiles them in userspace to more efficient executables which run in the kernel.

to expect the kernel maintainers to implement and formally verify an optimizing compiler.

A more realistic approach is for the kernel to trust the user to bring their own optimizer, and delegate the responsibility⁶. The user can trust an optimizer because of social convention (e.g., because it is provided by a trusted distributor) or because they have inspected the code. Eventually as formal verification becomes more widely adopted, they can switch over to formal proofs of correctness. This is no different than the reality of today: we trust kernel binaries because they were built by distributions we trust using compilers that the maintainers trust [52].

Proposal. Figure 8 shows the workflow we imagine. After an eBPF program is compiled and verified, if the user (1) has the correct system capability, (2) has registered an eBPF

⁶Only some users have the privilege to load an eBPF program. Similarly, only some can have the capability to use the optimizer.

optimizer with the kernel before, and (3) has requested optimization when loading the eBPF program, then the verified eBPF program is sent to the PVO running in the userspace, and the optimizer generates an executable in response. Otherwise, the eBPF program is passed to the existing JIT compiler for translation to native instructions.

5 Discussion

This proposal includes design decisions that require further discussion.

1. Why does the optimizer run in the userspace? Running PVO in the userspace allows reusing existing tools and libraries (e.g., by building on top of existing LLVM/BOLT tool [40]) for program analysis and optimization. The optimizer is not on the critical path and runs once per program execution, so the cost of kernel crossing is not an issue.

2. Can PVO apply missed compiler optimizations? The PVO operates late in the compilation pipeline, which may raise the question if it can find optimization opportunities that were missed in the first compilation phase. One solution is annotating eBPF programs to provide more context than just the program instructions. eBPF is already using *BPF Type Format* (BTF) for annotation and to enable the verifier to reason about the type of objects and function calls. The PVO tool can also expose a library of external functions (known as *kfunc* [15]) for developers to provide further information. The function call would be used by the PVO and ignored during code generation.

3. Can the verifier and the compiler improve to allow more optimizations? Verifier improvements are the focus of significant recent work [16, 20, 50, 51, 53–55], much of which has focused on reducing false positives (i.e., cases where the verifier rejects correct code). These improvements might allow several of the optimizations that are currently disallowed, however fundamental decidability limitations [26, 43] mean that they cannot allow all optimizations. PVO provides an alternative, that when combined with verifier improvements might allow much greater optimization.

Simultaneously, there has also been work on improving compilers [34, 56] for eBPF, though the focus is on automating the work of producing code that passes the verifier. These efforts do not generally improve optimality. Alternate approaches, including improvements to the JIT, have not been explored but could also improve optimality, and should be explored. However, PVO offers additional benefits: it makes it easier to explore and deploy new optimizations, including those that are program-specific (e.g., for networking code or targeting programs that work together [10]) or platform specific (e.g., for RISC-V). Furthermore, it also provides an easy experimental platform for testing optimization before they are implemented in the compiler or the kernel JIT.

4. Can PVO provide the same safety guarantees as the eBPF verifier? The current eBPF verifier checks the program being run, rather than the compiler that generated the bytecode. Thus, providing the same guarantees with PVO requires checking not just the optimizer but also the generated code. This is tricky because our goals make it so that the code generated by PVO might not be amenable to verification, and thus additional mechanisms are required to provide the same guarantees. Fortunately, several approaches could be used for this, including having the PVO generate certificates [20, 39, 51, 55] and sandboxing the executing code [13, 22, 23, 31, 33, 59]. Furthermore, approaches developed in the context of ensuring web browser safety when executing JavaScript might also be useful in this context. However, further investigation and work is necessary to apply any of these approaches in the PVO context. Finally, we note that our design assumes that a privileged user loads both optimizer and the program, and thus we rely on this users good judgment to avoid bugs. This is not unreasonable since a privileged user in Linux can violate desirable safety and security properties through a variety of alternate paths, even when the eBPF verifier is in use.

6 Related work

eBPF is widely used for performance-critical networking [3, 5, 17, 24, 29, 30, 35, 37, 58, 62, 63], storage [61, 65], and monitoring [6, 21], still prior work report performance degradation due to inefficient executables lacking support for hardware features. Miano *et al.* [35] and Paschali *et al.* [42] show that missing SIMD instructions slows hash computation and packet parsing. Shahinfar *et al.* [44] further note inefficient data copies and poor register utilization.

eBPF abstraction overheads also limit performance. Butrovich *et al.* [4] identify the map API as the bottleneck in their in-kernel database. Liu *et al.* [32] report cache misses in eBPF maps. Yang *et al.* [57] and Dwivedi *et al.* [13] optimized data structure abstractions, partially addressing the issue.

Previous compiler optimizations have focused on generating efficient verifier-compliant programs. Xu *et al.* [56] use heuristic search, while Mao *et al.* [34] adapt LLVM passes to eBPF constraints. Miano *et al.* [36] introduce profile-guided optimization (PGO) to restructure the code. Solleza *et al.* [48] introduce an SQL-like DSL that can benefit from a planer optimizing the application.

The eBPF runtime also has some code optimization. The kernel supports dead-code elimination since v5.1 [1], and there is an RFC for function inlining and branch elimination after verification [64]. Further, Zheng *et al.* [60] propose an extension that matches with code idioms and replaces them with more optimized variants that are not possible to generate in eBPF compiler pipeline. Their extension requires proof of equivalency of the new operation with the code it replaces to maintain correctness.

Post-verification optimization can also be applied to a collective of programs attached to same or various hooks in the kernel. Craun *et al.* [10] report a pair-wise pattern between eBPF observability programs and discuss benefits of joint-optimization in these cases. Kuo *et al.* [27] show that optimizing multiple programs attached to the same hook reduces eBPF runtime overhead.

Finally, there are proposals for changing how program verification and sandboxing is performed in eBPF. These proposal change the compilation pipeline and may relax limitations we mentioned in this work [22, 39, 51, 55].

7 Conclusion

In this work, we took a deep dive into the efficiency of generated eBPF executables and found that in certain cases, they are suboptimal. The cause is the tension between the compiler and the verifier. One tries to structure the code to improve performance, while the other requires certain structures to reason about the code's properties. We proposed a design that performs post-verification optimization and discussed its benefits and challenges. This design offers flexibility, enabling developers to make different trade-offs between performance and formal verification. But having it as part of the upstream kernel will be a battle to be fought, as the brave hearts of Scotland did.

Acknowledgments

We thank our shepherd, Tianyin Xu, and reviewers for their insightful comments. This work was supported by the European Union  under the grant CHORYS (Grant agreement ID: 101189551). The views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European Health and Digital Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

References

- [1] 2026. Dead Code Elimination: eBPF Documentation. <https://docs.ebpf.io/linux/concepts/verifier/#dead-code-elimination>.
- [2] Theophilus A. Benson, Prashanth Kannan, Prankur Gupta, Balasubramanian Madhavan, Kumar Saurabh Arora, Jie Meng, Martin Lau, Abhishek Dhamija, Rajiv Krishnamurthy, Srikanth Sundaresan, Neil Spring, and Ying Zhang. 2024. NetEdit: An Orchestration Platform for eBPF Network Functions at Scale. In *Special Interest Group on Data Communication (SIGCOMM)*. ACM.
- [3] Laurin Brandner, Ayush Mishra, Sebastiano Miano, Aurojit Panda, Gianni Antichi, and Laurent Vanbever. 2026. Offloading L7 Policies to the Kernel. [arXiv:2605.31084](https://arxiv.org/abs/2605.31084) [cs.NI].
- [4] Matthew Butrovich, Samuel Arch, Wan Shen Lim, William Zhang, Jignesh M Patel, and Andrew Pavlo. 2025. BPF-DB: A Kernel-Embedded Transactional Database Management System For eBPF Applications. In *Proceeding of ACM on Management of Data*, Volume: 3, Issue: 3. ACM.
- [5] Matthew Butrovich, Karthik Ramanathan, John Rollinson, Wan Shen Lim, William Zhang, Justine Sherry, and Andrew Pavlo. 2023. Tigger: A Database Proxy That Bounces with User-Bypass. In *Conference on Very Large Data Bases (VLDB)*, Volume: 16, Issue: 11. VLDB Endowment.
- [6] Cilium. [n. d.]. Tetragon, eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io>.
- [7] Cloud Native Computing Foundation. 2023. Cilium: Cloud native solution for networking, observability, and security. <https://cilium.io/>. Accessed: 2026-02-06.
- [8] Keith D. Cooper and Linda Torczon. 2022. *Engineering a Compiler* (3rd ed.). Morgan Kaufmann.
- [9] Jonathan Corbet. 2021. A different approach to BPF loops. <https://lwn.net/Articles/877062/>.
- [10] Milo Craun and Dan Williams. 2025. Pairwise BPF Programs Should Be Optimized Together. In *Proceedings of the 3rd Workshop on EBPF and Kernel Extensions (Coimbra, Portugal) (eBPF '25)*. Association for Computing Machinery, New York, NY, USA, 22–27. doi:10.1145/3748355.3748362
- [11] CrowdStrike. 2024. External Technical Root Cause Analysis – Channel File 291. <https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf>.
- [12] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. 2019. The Design and Operation of CloudLab. In *Annual Technical Conference (ATC)*. USENIX.
- [13] Kumar Kartikeya Dwivedi, Rishabh Iyer, and Sanidhya Kashyap. 2024. Fast, Flexible, and Practical Kernel Extensions. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (Austin, TX, USA) (SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 249–264. doi:10.1145/3694715.3695950
- [14] Yuqi Fu, Ruizhe Shi, Haoliang Wang, Songqing Chen, and Yue Cheng. 2024. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 19–36. <https://www.usenix.org/conference/atc24/presentation/fu>
- [15] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. 2024. The eBPF Runtime in the Linux Kernel. [arXiv:2410.00026](https://arxiv.org/abs/2410.00026) [cs.OS] <https://arxiv.org/abs/2410.00026>
- [16] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted Linux kernel extensions. In *Conference on Programming Language Design and Implementation (PLDI)*. ACM.
- [17] Yoann Ghigoff, Julien Sopena, Kahina Lazri, Antoine Blin, and Gilles Muller. 2021. BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing. In *Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX.
- [18] Brendan Gregg. 2024. No More Blue Fridays. <https://www.brendangregg.com/blog/2024-07-22/no-more-blue-fridays.html>.
- [19] LLVM User Guide. 2026. LLVM's Analysis and Transform Passes. <https://llvm.org/docs/Passes.html>.
- [20] Ameer Hamza, Lucas Zavalia, Arie Gurfinkel, Jorge A. Navas, and Grigory Fedyukovich. 2025. A Flow-Sensitive Refinement Type System for Verifying eBPF Programs. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 410 (Oct. 2025), 28 pages. doi:10.1145/3763799
- [21] Jörn-Thorben Hinz, Vamsi Addanki, Csaba Györgyi, Theo Jepsen, and Stefan Schmid. 2023. TCP's Third Eye: Leveraging eBPF for Telemetry-Powered Congestion Control. In *Workshop on EBPF and Kernel Extensions*. ACM.
- [22] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: closing the language-verifier gap with safe and usable kernel extensions. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference (Boston, MA, USA) (USENIX ATC '25)*. USENIX Association, USA, Article 20, 18 pages.
- [23] Di Jin, Alexander J. Gaidis, and Vasileios P. Kemerlis. 2024. Bee-Box: Hardening BPF against Transient Execution Attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 613–630. <https://www.usenix.org/conference/usenixsecurity24/presentation/jin-di>
- [24] Gyuyeong Kim and Dongsu Han. 2025. A Memory Pool Allocator for eBPF Applications. In *Workshop on EBPF and Kernel Extensions*. ACM.
- [25] Joanne Koong. 2022. BPF: Dynamic pointers. <https://lwn.net/Articles/895885/>.
- [26] Dexter C Kozen. 1977. Rice's theorem. In *Automata and Computability*. Springer, 245–248.
- [27] Hsuan-Chi Kuo, Kai-Hsun Chen, Yicheng Lu, Dan Williams, Sibin Mohan, and Tianyin Xu. 2022. Verified programs can party: optimizing kernel extensions via post-verification merging. In *European Conference on Computer Systems (EuroSys)*. ACM.
- [28] Xavier Leroy. 2009. Formal verification of a realistic compiler. In *Communications of the ACM*, Volume: 52, Issue: 7. ACM.
- [29] Tamás Lévai, Balázs Edvárd Kreith, and Gábor Rétvári. 2023. Supercharge WebRTC: Accelerate TURN Services with eBPF/XDP. In *Workshop on EBPF and Kernel Extensions*. ACM.
- [30] Shuo Li, Steven Chien, Tianyi Gao, and Michio Honda. 2026. Remote TCP Connection Offload and Applications. In *Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX.
- [31] Soo Yee Lim, Tanya Prasad, Xueyuan Han, and Thomas Pasquier. 2024. SafeBPF: Hardware-assisted Defense-in-depth for eBPF Kernel Extensions. In *Cloud Computing Security Workshop (CCSW)*. ACM.
- [32] Chang Liu, Byungchul Tak, and Long Wang. 2024. Understanding Performance of eBPF Maps. In *Workshop on EBPF and Kernel Extensions*. ACM.
- [33] Hongyi Lu, Shuai Wang, Yechang Wu, Wanning He, and Fengwei Zhang. 2024. MOAT: Towards Safe BPF Kernel Extension. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 1153–1170. <https://www.usenix.org/conference/usenixsecurity24/presentation/lu-hongyi>

- [34] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. 2024. Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.
- [35] Sebastiano Miano, Xiaoqi Chen, Ran Ben Basat, and Gianni Antichi. 2023. Fast In-kernel Traffic Sketching in eBPF. In *SIGCOMM Computer Communication Review (CCR)*, Volume: 53, Issue: 1. ACM.
- [36] Sebastiano Miano, Alireza Sanaee, Fulvio Risso, Gábor Rétvári, and Gianni Antichi. 2024. Morpheus: A Run Time Compiler and Optimizer for Software Data Planes. In *Transactions on Networking*, Volume: 3, Issue: 3. IEEE/ACM.
- [37] Máté Nagy, Tamás Lévai, Felician Németh, Aurojit Panda, Gianni Antichi, and Gábor Rétvári. 2026. Elastic Scaling of Real-Time Communication Services. In *IEEE Transactions on Network and Service Management*, Volume: 23. IEEE.
- [38] Andrii Nakryiko. 2023. BPF open-coded iterators. <https://lwn.net/Articles/925751/>.
- [39] George C. Necula and Peter Lee. 1996. Safe Kernel Extensions Without Run-Time Checking. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX.
- [40] Maksim Panchenko, Rafael Auler, Bill Nell, and Guilherme Ottoni. 2019. BOLT: A Practical Binary Optimizer for Data Centers and Beyond. In *International Symposium on Code Generation and Optimization (CGO)*. IEEE/ACM.
- [41] Aurojit Panda, Sangjin Han, Keon Jang, Melvin Walls, Sylvia Ratnasamy, and Scott Shenker. 2016. NetBricks: Taking the V out of NFV. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX.
- [42] Vladimiro Paschali, Andrea Monterubbiano, Francesco Faz-zari, Sebastiano Miano, and Salvatore Pontarelli. 2025. X2DP: eXtended eXpress Data Path. In *Proceedings of the ACM SIGCOMM 2025 Posters and Demos (Coimbra, Portugal) (ACM SIGCOMM Posters and Demos '25)*. Association for Computing Machinery, New York, NY, USA, 132–133. doi:10.1145/3744969.3748399
- [43] H. G. Rice. 1953. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*, Issue: 2, Volume: 74.
- [44] Farbod Shahinfar, Sebastiano Miano, Aurojit Panda, and Gianni Antichi. 2025. Demystifying Performance of eBPF Network Applications. In *International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. ACM.
- [45] Farbod Shahinfar, Sebastiano Miano, Alireza Sanaee, Giuseppe Siracusano, Roberto Bifulco, and Gianni Antichi. 2021. The case for network functions decomposition. In *International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. ACM.
- [46] Nikita Shirokov and Ranjeeth Dasineni. 2018. Katran, a scalable network load balancer. <https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran-a-scalable-network-load-balancer/>.
- [47] Jakub Sitnicki. 2019. eBPF can't count?! <https://blog.cloudflare.com/ebpf-cant-count/>.
- [48] Franco Solleza, Justus Adam, Akshay Narayan, Malte Schwarzkopf, Andrew Crotty, and Nesime Tatbul. 2025. Kernel Extension DSLs Should Be Verifier-Safe!. In *Workshop on EBPF and Kernel Extensions*. ACM.
- [49] Alexei Starovoitov. 2024. BPF Arena. <https://lwn.net/Articles/961594/>.
- [50] Hao Sun and Zhendong Su. 2024. Validating the eBPF Verifier via State Embedding. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX.
- [51] Hao Sun and Zhendong Su. 2025. Prove It to the Kernel: Precise Extension Analysis via Proof-Guided Abstraction Refinement. In *Symposium on Operating Systems Principles (SOSP)*. ACM.
- [52] Ken Thompson. 1984. Reflections on trusting trust. In *Communications of the ACM*, Volume: 27, Issue: 8. ACM.
- [53] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 254–265. doi:10.1109/CGO53902.2022.9741267
- [54] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification (CAV)*. Springer-Verlag.
- [55] Xiwei Wu, Yueyang Feng, Tianyi Huang, Xiaoyang Lu, Shengkai Lin, Lihan Xie, Shizhen Zhao, and Qinxiang Cao. 2025. VEP: A Two-stage Verification Toolchain for Full eBPF Programmability. In *Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX.
- [56] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *Special Interest Group on Data Communication (SIGCOMM)*. ACM.
- [57] Bin Yang, Dian Shen, Junxue Zhang, Hanlin Yang, Lunqi Zhao, Beilun Wang, Guyue Liu, and Kai Chen. 2025. eNetSTL: Towards an In-kernel Library for High-Performance eBPF-based Network Functions. In *European Conference on Computer Systems (EuroSys)*. ACM.
- [58] Rui Yang and Marios Kogias. 2023. HEELS: A Host-Enabled eBPF-Based Load Balancing Scheme. In *Workshop on EBPF and Kernel Extensions*. ACM.
- [59] Peihua Zhang, Chenggang Wu, Xiangyu Meng, Yinqian Zhang, Mingfan Peng, Shiyang Zhang, Bing Hu, Mengyao Xie, Yuanming Lai, Yan Kang, and Zhe Wang. 2024. HIVE: A Hardware-assisted Isolated Execution Environment for eBPF on AArch64. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 163–180. <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-peihua>
- [60] Yusheng Zheng, Zhengjie Ji, Weichen Tao, Hao Sun, Wei Zhang, Dan Williams, and Andi Quinn. 2026. Kops: Safely Extending the eBPF Compilation Pipeline with Native Operations. arXiv:2606.24213 [cs.OS] <https://arxiv.org/abs/2606.24213>
- [61] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX.
- [62] Yang Zhou, Zezhou Wang, Sowmya Dharanipragada, and Minlan Yu. 2023. Electrode: Accelerating Distributed Protocols with eBPF. In *Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX.
- [63] Yang Zhou, Xingyu Xiang, Matthew Kiley, Sowmya Dharanipragada, and Minlan Yu. 2024. DINT: Fast In-Kernel Distributed Transactions with eBPF. In *Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX.
- [64] Eduard Zingerman. 2024. bpf: inlinable kfuncs for BPF. <https://lwn.net/Articles/997401/>.
- [65] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. 2025. cache_ext: Customizing the Page Cache with eBPF. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. Association for Computing Machinery. doi:10.1145/3731569.3764820